

Exercise 7

Introduction to MySQL — Beginner Lab Manual

What's inside

7(a)	Create a database and a table (MySQL Command Line Client)
7(b)	Create a table, insert data and update data
7(c)	Subqueries (a query inside another query)
7(d)	Create and run script files in MySQL Workbench
7(e)	Database folder + init.sql file integrated into an API

Before you start (do this once)

1. Install **MySQL Server** and **MySQL Workbench** (MySQL Installer for Windows).
Remember the **root password** you set.
2. Open **Start menu > MySQL Command Line Client** and type your password.
3. When you see `mysql>` you are ready.

Rules to remember

- Every SQL command ends with a semicolon ;
- SQL keywords are not case-sensitive (SELECT = select), but we write them in CAPITALS so they are easy to read.
- Text values go inside single quotes: 'Ravi'. Numbers do not.

Common data types

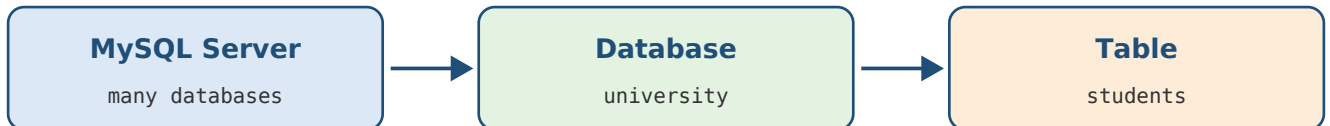
Type	Stores	Example
INT	Whole numbers	85
VARCHAR(50)	Text up to 50 characters	'Ravi'
DATE	A date	'2026-10-07'

Program 7(a): Create a Database and a Table

Using the MySQL Command Line Client

AIM: To create a database and a table inside that database using the MySQL Command Line Client.

Idea



Queries

MYSQL COMMAND LINE CLIENT

```
CREATE DATABASE university;      -- 1. make a new database
SHOW DATABASES;                  -- 2. list all databases
USE university;                  -- 3. open (select) it

CREATE TABLE students (         -- 4. make a table
  id INT PRIMARY KEY,
  name VARCHAR(50),
  dept VARCHAR(10),
  marks INT
);

SHOW TABLES;                   -- 5. list tables
DESC students;                   -- 6. show table structure
```

Output

OUTPUT

```
mysql> CREATE DATABASE university;
Query OK, 1 row affected (0.01 sec)
```

```
mysql> SHOW DATABASES;
```

```
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
| university |
+-----+
```

```
mysql> USE university;
Database changed
```

```
mysql> SHOW TABLES;
```

```
+-----+
| Tables_in_university |
+-----+
| students |
+-----+
```

```
mysql> DESC students;
```

```
+-----+-----+-----+-----+-----+-----+
| Field | Type      | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int       | NO   | PRI | NULL    |      |
| name  | varchar(50) | YES  |     | NULL    |      |
| dept  | varchar(10) | YES  |     | NULL    |      |
| marks | int       | YES  |     | NULL    |      |
+-----+-----+-----+-----+-----+-----+
```

TIP: PRIMARY KEY means every student must have a unique id. Two students cannot have the same id.

Result: The database university and the table students were created successfully.

Program 7(b): Create, Insert and Update

Work with a 'teachers' table

AIM: To write MySQL queries to create a table, insert data and update data in the table.

Queries

MYSQL

```
USE university;
```

```
-- 1. CREATE the table
CREATE TABLE teachers (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  subject VARCHAR(30),
  salary INT
);
```

```
-- 2. INSERT data (3 rows)
INSERT INTO teachers VALUES
(1, 'Ramesh', 'Java', 40000),
(2, 'Lakshmi', 'DBMS', 45000),
(3, 'Suresh', 'Python', 42000);
```

```
SELECT * FROM teachers;      -- see the data
```

```
-- 3. UPDATE: change Lakshmi's salary
UPDATE teachers SET salary = 50000 WHERE id = 2;
```

```
SELECT * FROM teachers;      -- see the change
```

Output

OUTPUT

```
mysql> INSERT INTO teachers VALUES ...
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM teachers;      -- BEFORE update
+----+-----+-----+-----+
| id | name   | subject | salary |
+----+-----+-----+-----+
| 1  | Ramesh | Java    | 40000  |
| 2  | Lakshmi | DBMS    | 45000  |
| 3  | Suresh | Python  | 42000  |
+----+-----+-----+-----+

mysql> UPDATE teachers SET salary = 50000 WHERE id = 2;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM teachers;      -- AFTER update
+----+-----+-----+-----+
| id | name   | subject | salary |
+----+-----+-----+-----+
| 1  | Ramesh | Java    | 40000  |
| 2  | Lakshmi | DBMS    | 50000  |
| 3  | Suresh | Python  | 42000  |
+----+-----+-----+-----+
```

WARNING: Always use **WHERE** with UPDATE. Without it, **every row** in the table gets changed!

Result: The table was created, data was inserted and updated successfully.

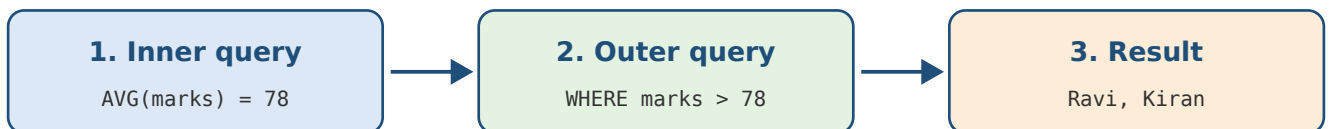
Program 7(c): Subqueries

A query written inside another query

AIM: To write MySQL queries to implement subqueries in the MySQL Command Line Client.

How a subquery works

The **inner query** (in brackets) runs **first**. Its answer is then used by the **outer query**.



Step 1: Add data to the students table (from 7a)

MYSQL

```
USE university;
```

```
INSERT INTO students VALUES  
(1, 'Ravi', 'CSE', 85),  
(2, 'Sita', 'ECE', 72),  
(3, 'Kiran', 'CSE', 90),  
(4, 'Anu', 'EEE', 65),  
(5, 'Rahul', 'ECE', 78);
```

Step 2: Subqueries

MYSQL

```
-- Q1. Students who scored MORE than the average  
SELECT name, marks FROM students  
WHERE marks > (SELECT AVG(marks) FROM students);  
  
-- Q2. Student with the HIGHEST marks  
SELECT name, marks FROM students  
WHERE marks = (SELECT MAX(marks) FROM students);  
  
-- Q3. Students in the SAME department as Ravi  
SELECT name, dept FROM students  
WHERE dept = (SELECT dept FROM students WHERE name = 'Ravi');
```

Output

OUTPUT

Q1. (inner query: average = 78)

name	marks
Ravi	85
Kiran	90

Q2. (inner query: maximum = 90)

name	marks
Kiran	90

Q3. (inner query: Ravi's dept = CSE)

name	dept
Ravi	CSE
Kiran	CSE

Result: Subqueries were executed successfully in the MySQL Command Line Client.

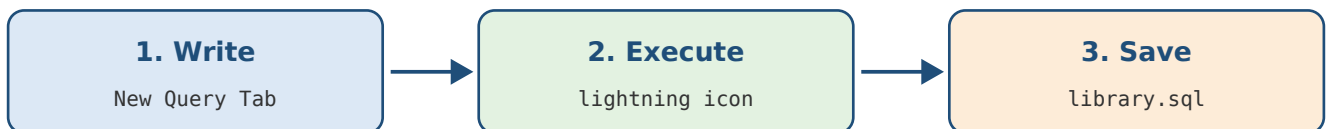
Program 7(d): Script Files in MySQL Workbench

Save many SQL commands in one .sql file and run them together

AIM: To create and run SQL script files in MySQL Workbench.

What is a script file?

A **script file** is a text file ending in .sql that stores many SQL commands. You can save it, share it, and run it again anytime instead of typing everything again.



Steps

1. Open **MySQL Workbench** and click **Local instance MySQL**. Enter your password.
2. Go to **File > New Query Tab**.
3. Type the script below.
4. Click the **lightning icon** (Execute) to run the whole script.
5. Go to **File > Save Script As** and save it as library.sql.
6. To use it later: **File > Open SQL Script**, choose the file and click Execute.

Script (library.sql)

library.sql

```
-- library.sql : my first script file
CREATE DATABASE IF NOT EXISTS library;
USE library;

CREATE TABLE IF NOT EXISTS books (
  id INT PRIMARY KEY AUTO_INCREMENT,
  title VARCHAR(100),
  author VARCHAR(50)
);

INSERT INTO books (title, author) VALUES
('Wings of Fire', 'A.P.J. Abdul Kalam'),
('Let Us C', 'Yashavant Kanetkar');

SELECT * FROM books;
```

Output (Result Grid in Workbench)

OUTPUT

```
+-----+-----+-----+
| id | title          | author          |
+-----+-----+-----+
| 1  | Wings of Fire | A.P.J. Abdul Kalam |
| 2  | Let Us C      | Yashavant Kanetkar |
+-----+-----+-----+
```

TIP: IF NOT EXISTS stops errors if the database/table already exists.

AUTO_INCREMENT gives ids 1, 2, 3... automatically. You can also run the file from the command line: `SOURCE C:/sql/library.sql;`

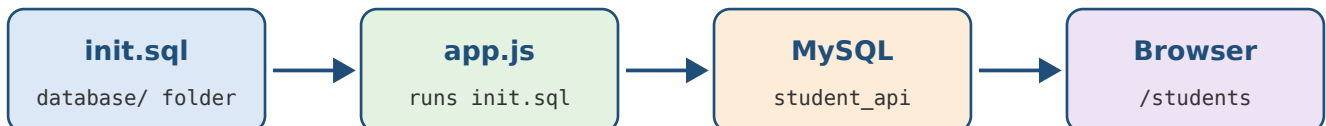
Result: A SQL script file was created, saved and executed in MySQL Workbench.

Program 7(e): Database Folder + init.sql in an API

Keep the database setup inside the project and run it from Node.js

AIM: To create a database directory in a project, initialize a database using an init.sql file, and integrate it into an Express API.

How it works



Step 1: Create the project

TERMINAL

```
mkdir student-api
cd student-api
mkdir database
npm init -y
npm install express mysql2
```

Project structure

FOLDERS

```
student-api/
├── database/
│   └── init.sql      <- database setup file
├── node_modules/
├── app.js           <- the API
└── package.json
```

Step 2: database/init.sql

database/init.sql

```
CREATE DATABASE IF NOT EXISTS student_api;
USE student_api;

CREATE TABLE IF NOT EXISTS students (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  dept VARCHAR(10)
);

-- IGNORE skips rows that already exist (safe to run many times)
INSERT IGNORE INTO students VALUES
(1, 'Ravi', 'CSE'),
(2, 'Sita', 'ECE');
```

Step 3: app.js

app.js

```
const express = require('express');
const mysql = require('mysql2');
const fs = require('fs');           // to read files
const app = express();

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'your_password',
  multipleStatements: true          // run the whole .sql file at once
});

// Read init.sql and run it to set up the database
const sql = fs.readFileSync('./database/init.sql', 'utf8');
db.query(sql, (err) => {
  if (err) return console.log('Setup failed: ' + err.message);
  console.log('Database initialized from init.sql');
});

// API route: get all students
app.get('/students', (req, res) => {
  db.query('SELECT * FROM students', (err, rows) => {
    if (err) return res.status(500).send(err.message);
    res.json(rows);
  });
});

app.listen(3000, () => console.log('API running at http://localhost:3000'));
```

TERMINAL (run inside the student-api folder)

```
node app.js
```

OUTPUT - TERMINAL

```
API running at http://localhost:3000
Database initialized from init.sql
```

OUTPUT - BROWSER

```
Browser at http://localhost:3000/students
```

```
[{"id":1,"name":"Ravi","dept":"CSE"}, {"id":2,"name":"Sita","dept":"ECE"}]
```

NOTE: No database name is given in createConnection because init.sql creates it and selects it with USE student_api;. You can also run init.sql manually in Workbench (File > Open SQL Script).

Result: A database directory with an init.sql file was created and integrated into the Express API successfully.